MAT41-3

NN & Deep Learning Math

Forward propagation, backpropagation, softmax, and the dynamics of training

---

A neural network is a long composition of affine maps and nonlinearities — and everything it does, including how it learns, is the chain rule applied carefully.

Albert School — 30 hours

Preface

You arrive at this course already holding every tool it needs; what is new is the way they combine. From **Foundations** (MAT11-1) you bring the **chain rule** — the single most important fact in all of deep learning — together with Taylor's first-order approximation and the idea of convexity. From **Linear Algebra II** (MAT21-1) you bring matrices as linear maps, matrix products, transposes, and the eigenvalue picture that will explain why deep networks are so hard to train. From **Linear Algebra III** (MAT31-1) you bring the scalar product, the outer product, symmetric and positive-definite matrices, and the Hessian's role in classifying critical points. From **Probability & Statistics II** (MAT31-2) you bring probability distributions, expectation, and **maximum likelihood**, which is where the loss function we minimize will come from.

A **feedforward neural network** is, stripped of mystique, a function built by alternating two operations you already understand: an **affine map** $z = Wa + b$ and a **component-wise nonlinearity** $a = \sigma(z)$. Stack a few dozen of these and you have a function with millions of tunable numbers. The two questions of the course are exactly the two questions you would ask of any such function:

- **How does it compute?** Running the layers front to back is **forward propagation** — nothing more than evaluating a composition.
- **How does it learn?** Adjusting the millions of parameters to reduce a loss requires the gradient of that loss with respect to every parameter. Computing that gradient efficiently is **backpropagation** — nothing more than the chain rule organized so that no derivative is ever computed twice.

Around this core we will assemble the rest of the modern toolkit: the **softmax** that turns raw scores into probabilities, the **cross-entropy** loss that maximum likelihood hands us, and the structural tricks — **residual connections**, **layer normalization**, **careful initialization**, **weight decay**, **dropout** — that decide whether training succeeds or collapses. Throughout, resist the urge to see anything magical. A neural network is a composition; its gradient is the chain rule; its difficulties are the eigenvalues of repeated multiplication. Keep that slogan in view and the entire subject organizes itself.

A note on notation. We write $a^{(0)} = x$ for the input, and for each layer $l = 1, ..., L$ we write $W^{(l)}$ and $b^{(l)}$ for its weight matrix and bias vector, $z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}$ for its **pre-activation**, and $a^{(l)} = \sigma\big(z^{(l)}\big)$ for its **activation**. The final activation $\hat{y} = a^{(L)}$ is the network's prediction. The Hadamard (component-wise) product is written $\odot$.

Contents

1 Forward propagation

A neural network computes by passing a vector through a sequence of **layers**. We first watch a small network compute on an actual input, by hand, and only then state the general rule. The point to absorb is that there is nothing to a forward pass but **matrix–vector multiply, add a bias, apply a function, repeat**.

1.1 A network computing, by hand

Consider a tiny network with two inputs, one hidden layer of two units, and one output. Its parameters are

$$W^{(1)} = \begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix}, \quad b^{(1)} = \begin{pmatrix} 0 \\ -1 \end{pmatrix}, \quad W^{(2)} = \begin{pmatrix} 2 & 1 \end{pmatrix}, \quad b^{(2)} = 1,$$

and the hidden layer uses the **ReLU** activation $\sigma(t) = \max(t, 0)$, applied component-wise. Feed in $x = (1, 2)$.

Worked example — one forward pass. Layer 1. Form the affine map, then activate:

$$z^{(1)} = W^{(1)}x + b^{(1)} = \begin{pmatrix} 1 \cdot 1 + (-1) \cdot 2 \\ 1 \cdot 1 + 1 \cdot 2 \end{pmatrix} + \begin{pmatrix} 0 \\ -1 \end{pmatrix} = \begin{pmatrix} -1 \\ 3 \end{pmatrix} + \begin{pmatrix} 0 \\ -1 \end{pmatrix} = \begin{pmatrix} -1 \\ 2 \end{pmatrix},$$

$$a^{(1)} = \sigma\big(z^{(1)}\big) = \begin{pmatrix} \max(-1, 0) \\ \max(2, 0) \end{pmatrix} = \begin{pmatrix} 0 \\ 2 \end{pmatrix}.$$

Notice the first unit's pre-activation was negative, so ReLU **switched it off**.

Layer 2. The output layer here is purely affine (identity activation):

$$z^{(2)} = W^{(2)}a^{(1)} + b^{(2)} = 2 \cdot 0 + 1 \cdot 2 + 1 = 3, \quad \hat{y} = 3.$$

The whole computation is three multiply–add steps and one thresholding. That is a forward pass.

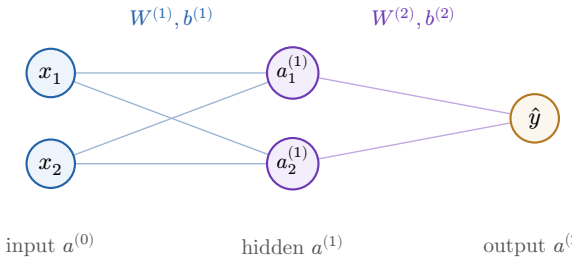

Figure 1: The 2–2–1 network of the worked example. Each edge carries a weight; each non-input node adds a bias and applies its activation. Information flows strictly left to right in a forward pass.

1.2 The general rule

The example generalizes verbatim. A layer is an affine map followed by a component-wise function.

Definition 1 (Layer) Layer l has weight matrix $W^{(l)} \in \mathbb{R}^{n_l \times n_{l-1}}$ and bias $b^{(l)} \in \mathbb{R}^{n_l}$. Given the previous activation $a^{(l-1)}$, it computes the **pre-activation**

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)} \in \mathbb{R}^{n_l}$$

and then the **activation** (post-activation)

$$a^{(l)} = \sigma\big(z^{(l)}\big), \quad \text{i.e.} \quad \big(a^{(l)}\big)_i = \sigma\Big(\big(z^{(l)}\big)_i\Big),$$

with the scalar function σ applied to each component separately.

Definition 2 (Forward pass) Given input x, the **forward pass** of an L-layer network is the composition

$$a^{(0)} = x, \quad z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \quad a^{(l)} = \sigma\big(z^{(l)}\big) \quad (l = 1, ..., L),$$

with prediction $\hat{y} = a^{(L)}$.

Remark The shape bookkeeping is the whole content of "by hand". Layer l turns a vector of length n_{l-1} into one of length n_l, so $W^{(l)}$ is $n_l \times n_{l-1}$ — **rows index the layer you are entering, columns the layer you came from**. Getting these dimensions to chain is exactly the matrix-product compatibility you learned in MAT21-1.

Common pitfall The activation is applied **component-wise**, not as a matrix or vector operation. $\sigma(z)$ does not mix the components of z together — unit i of the output depends only on z_i. (The softmax of the next chapters is the one deliberate exception, and we will flag it as such.) Confusing a component-wise σ with a vector-valued map is the root of most early backpropagation errors.

1.3 The activation zoo

The choice of σ matters, but every common activation is a simple scalar function. Three appear constantly.

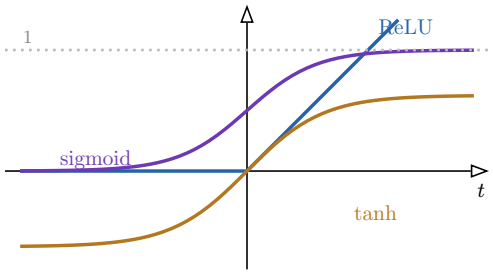

Figure 2: Three standard activations. **ReLU** $\sigma(t) = \max(t, 0)$ is linear for $t > 0$ and flat for $t < 0$ (so σ' is 1 or 0). **Sigmoid** $\sigma(t) = 1/(1 + e^{-t})$ squashes into $(0, 1)$; **tanh** into $(-1, 1)$. Both saturate — their slopes flatten toward 0 far from the origin, a fact that will return as the vanishing-gradient problem.

Remark ReLU's derivative is the cleanest imaginable: $\sigma'(t) = 1$ for $t > 0$ and 0 for $t < 0$ (the kink at 0 is conventionally assigned 0). The sigmoid and tanh derivatives, by contrast, are largest at the origin and decay to 0 in the tails — they **saturate**. Keep this in mind; it is the seed of a problem we diagnose in Chapter 8.

Exercises

1. Run a forward pass of the worked-example network on $x = (2, 0)$. Which hidden unit is switched off by ReLU?

2. A network has layer sizes $3 \to 5 \to 5 \to 2$. State the dimensions of each $W^{(l)}$ and $b^{(l)}$, and confirm the products chain.

3. Compute $\sigma'(t)$ for the sigmoid and show it equals $\sigma(t)(1 - \sigma(t))$. Where is it largest?

4. With the worked-example weights but a sigmoid output activation, recompute $\hat{y}$ for $x = (1, 2)$.

2 Why activations matter

It is tempting to think depth alone — many layers — is what makes a network powerful. It is not. **Without the nonlinear activations, depth buys nothing at all.** We prove this precisely, because it is the cleanest possible justification for why σ is not optional.

2.1 Stacking linear maps collapses

Worked example — two linear layers are one. Take two layers with identity activation: $a^{(1)} = W^{(1)}x + b^{(1)}$ and $a^{(2)} = W^{(2)}a^{(1)} + b^{(2)}$. Substitute the first into the second:

$$a^{(2)} = W^{(2)}\big(W^{(1)}x + b^{(1)}\big) + b^{(2)} = \underbrace{\big(W^{(2)}W^{(1)}\big)}_{W}\,x + \underbrace{\big(W^{(2)}b^{(1)} + b^{(2)}\big)}_{b}.$$

The two layers together compute $Wx + b$ — a **single** affine map. The product $W^{(2)}W^{(1)}$ is just some matrix; the network gained no expressive power from having two layers instead of one.

2.2 The general collapse

Proposition 3 (Linear collapse) If every activation is the identity, $\sigma = \mathrm{id}$, then an L-layer network computes a single affine map

$$a^{(L)} = Wx + b, \quad \text{with} \quad W = W^{(L)}W^{(L-1)}\ldots W^{(1)}$$

and b a corresponding combination of the biases.

Proof. With $\sigma = \mathrm{id}$, each layer is $a^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}$. Substituting layer by layer, as in the example,

$$a^{(2)} = W^{(2)}W^{(1)}a^{(0)} + \big(W^{(2)}b^{(1)} + b^{(2)}\big),$$

and an easy induction carries this to layer L: the weight matrices accumulate into the single product $W = W^{(L)}\ldots W^{(1)}$, while the biases accumulate into a single vector b. The output is $Wx + b$. □

A product of matrices is one matrix, so an arbitrarily deep network **with no nonlinearities** represents only the affine functions a single layer already represents. Depth is wasted. The nonlinear σ between layers is precisely what prevents this collapse and lets composition build genuinely richer functions — it is the difference between drawing a single straight cut through your data and folding the space first.

Common pitfall The collapse needs **every** activation to be linear. A single nonlinear layer anywhere breaks the matrix-product argument, because $\sigma(Wx+b)$ cannot be pulled out as a matrix times a vector. This is why even one hidden nonlinearity already lifts a network beyond linear models.

Exercises
5. For $W^{(1)} = \begin{pmatrix} 1 & 2 \\ 0 & 1 \end{pmatrix}$, $W^{(2)} = \begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix}$ and zero biases with identity activations, compute the single matrix W the two layers represent.
6. Explain why a network with identity activations, however deep, can never represent the function $f(x) = x^2$ on $\mathbb{R}$.
7. Show that if all activations are the **same** fixed linear function $\sigma(t) = ct$ (not necessarily the identity), the network still collapses to an affine map. What plays the role of W?

3 Backpropagation

Training adjusts the parameters to reduce a scalar **loss** L computed from the output $\hat{y} = a^{(L)}$ and the target y. Gradient-based training needs $\partial L/\partial W^{(l)}$ and $\partial L/\partial b^{(l)}$ for **every** layer. Computing each of these from scratch would repeat enormous amounts of work; **backpropagation** is the bookkeeping that computes them all in a single backward sweep, reusing shared sub-derivatives. It is the chain rule, organized.

3.1 The key quantity: the error vector

The trick is to not chase $\partial L/\partial W^{(l)}$ directly, but first to compute a more fundamental object at each layer: the sensitivity of the loss to that layer's pre-activation.

Definition 4 (Error vector) The **error** (or **delta**) of layer l is

$$\delta^{(l)} = \frac{\partial L}{\partial z^{(l)}} \in \mathbb{R}^{n_l}, \quad \left(\delta^{(l)}\right)_i = \frac{\partial L}{\partial \left(z^{(l)}\right)_i}.$$

It measures how much the loss would change per unit change in each component of the pre-activation $z^{(l)}$.

Once the deltas are known, the parameter gradients fall out immediately (Section 3.4). So the whole problem reduces to computing $\delta^{(L)}, \delta^{(L-1)}, \ldots, \delta^{(1)}$ — and these satisfy a backward recurrence.

3.2 The two backpropagation equations

Start at the output, where the loss is defined, then propagate backward.

Proposition 5 (Output error)

$$\delta^{(L)} = \frac{\partial L}{\partial a^{(L)}} \odot \sigma'\left(z^{(L)}\right),$$

the loss gradient with respect to the output, gated component-wise by the output layer's activation slope.

Proposition 6 (Backward recurrence) For $l = L-1, \ldots, 1$,

$$\delta^{(l)} = \left(\left(W^{(l+1)}\right)^T \delta^{(l+1)}\right) \odot \sigma'\left(z^{(l)}\right).$$

Proof. By the chain rule through the pre-activation of the next layer, $\delta^{(l)} = \left(\partial z^{(l+1)}/\partial z^{(l)}\right)^T \delta^{(l+1)}$. Now $z^{(l+1)} = W^{(l+1)}\sigma\left(z^{(l)}\right) + b^{(l+1)}$, so its Jacobian with respect to $z^{(l)}$ is $W^{(l+1)}\,\mathrm{diag}\left(\sigma'\left(z^{(l)}\right)\right)$. Transposing,

$$\left(\partial \frac{z^{(l+1)}}{\partial} z^{(l)}\right)^T = \mathrm{diag}\left(\sigma'\left(z^{(l)}\right)\right)\left(W^{(l+1)}\right)^T,$$

and multiplying by $\delta^{(l+1)}$ gives exactly $\left(\left(W^{(l+1)}\right)^T \delta^{(l+1)}\right) \odot \sigma'\left(z^{(l)}\right)$, the diagonal matrix acting as a component-wise multiply by $\sigma'\left(z^{(l)}\right)$. □

The structure is worth reading aloud: to push the error back one layer, **multiply by the transposed weight matrix**, then **gate by the local activation slope**. The transpose is not a coincidence — the forward pass multiplies by $W^{(l+1)}$ to go from layer l to layer $l+1$; sensitivity flows the opposite way, and the adjoint of "multiply by W" is "multiply by W^T".

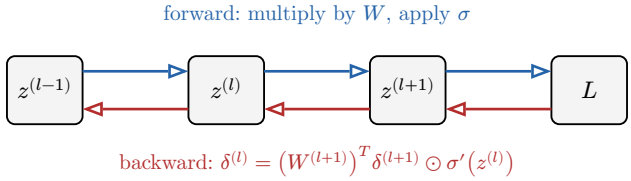

Figure 3: The computational graph and its reverse. The forward pass (blue, top) multiplies by weights and applies activations left to right. Backpropagation (red, bottom) carries the error δ right to left through the **transposed** weights, gating by the local activation slope at each step.

3.3 Parameter gradients are outer products

With the deltas in hand, the gradients we actually want are one short step away.

Proposition 7 (Weight and bias gradients)

$$\frac{\partial L}{\partial W^{(l)}} = \delta^{(l)}\left(a^{(l-1)}\right)^T, \quad \frac{\partial L}{\partial b^{(l)}} = \delta^{(l)}.$$

The weight gradient is the **outer product** of the layer's error with its incoming activation.

Proof. Since $\left(z^{(l)}\right)_i = \sum_k \left(W^{(l)}\right)_{ik}\left(a^{(l-1)}\right)_k + \left(b^{(l)}\right)_i$, we have $\partial\left(z^{(l)}\right)_i / \partial\left(W^{(l)}\right)_{ik} = \left(a^{(l-1)}\right)_k$. The chain rule gives

$$\frac{\partial L}{\partial\left(W^{(l)}\right)_{ik}} = \left(\delta^{(l)}\right)_i \left(a^{(l-1)}\right)_k,$$

which is exactly the (i,k) entry of the outer product $\delta^{(l)}\left(a^{(l-1)}\right)^T$. For the bias, $\partial\left(z^{(l)}\right)_i / \partial\left(b^{(l)}\right)_i = 1$, so $\partial L/\partial b^{(l)} = \delta^{(l)}$. □

Remark The outer product $\delta^{(l)}\big(a^{(l-1)}\big)^T$ (a column times a row) is an $n_l \times n_{l-1}$ matrix — the same shape as $W^{(l)}$, as it must be. This is the rank-one matrix from MAT31-1 reappearing as the elementary building block of every weight update.

Backpropagation, end to end.

1. **Forward.** Run the forward pass, **storing** every $z^{(l)}$ and $a^{(l)}$.
2. **Output error.** $\delta^{(L)} = \big(\partial L/\partial a^{(L)}\big) \odot \sigma'\big(z^{(L)}\big)$.
3. **Backward.** For $l = L-1, \ldots, 1$: $\delta^{(l)} = \Big(\big(W^{(l+1)}\big)^T \delta^{(l+1)}\Big) \odot \sigma'\big(z^{(l)}\big)$.
4. **Read off gradients.** $\partial L/\partial W^{(l)} = \delta^{(l)}\big(a^{(l-1)}\big)^T$ and $\partial L/\partial b^{(l)} = \delta^{(l)}$.

3.4 Backpropagation, all the way through, by hand

We now backpropagate through the exact network of Chapter 1. Recall $x = (1, 2)$ gave $z^{(1)} = (-1, 2)$, $a^{(1)} = (0, 2)$, and $\hat{y} = 3$. Train it on target $y = 1$ with squared-error loss $L = \frac{1}{2}(\hat{y} - y)^2$, and an identity output activation.

Worked example — a full backward pass. Loss and output error. $L = \frac{1}{2}(3-1)^2 = 2$. The output is affine, so $\sigma'\big(z^{(2)}\big) = 1$ and

$$\delta^{(2)} = \frac{\partial L}{\partial a^{(2)}} \cdot 1 = (\hat{y} - y) = 2.$$

Layer-2 gradients.

$$\frac{\partial L}{\partial W^{(2)}} = \delta^{(2)}\big(a^{(1)}\big)^T = 2 \cdot (0, 2) = (0, 4), \quad \frac{\partial L}{\partial b^{(2)}} = 2.$$

Backpropagate to layer 1. First push through the transposed weights, then gate by ReLU's slope. Here $\sigma'\big(z^{(1)}\big) = \sigma'((-1, 2)) = (0, 1)$ (off where the pre-activation was negative):

$$\big(W^{(2)}\big)^T \delta^{(2)} = \begin{pmatrix} 2 \\ 1 \end{pmatrix} \cdot 2 = \begin{pmatrix} 4 \\ 2 \end{pmatrix}, \quad \delta^{(1)} = \begin{pmatrix} 4 \\ 2 \end{pmatrix} \odot \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 0 \\ 2 \end{pmatrix}.$$

Layer-1 gradients.

$$\frac{\partial L}{\partial W^{(1)}} = \delta^{(1)} x^T = \begin{pmatrix} 0 \\ 2 \end{pmatrix} (1, 2) = \begin{pmatrix} 0 & 0 \\ 2 & 4 \end{pmatrix}, \quad \frac{\partial L}{\partial b^{(1)}} = \begin{pmatrix} 0 \\ 2 \end{pmatrix}.$$

Every gradient the optimizer needs, from one forward pass and one backward sweep. Note how the dead ReLU unit ($\sigma' = 0$) zeroed out the entire first row of $\partial L/\partial W^{(1)}$: a unit that did not fire receives no gradient.

Common pitfall You **must** store the forward-pass values $z^{(l)}$ and $a^{(l)}$ — the backward pass needs $\sigma'\big(z^{(l)}\big)$ and $a^{(l-1)}$ at every layer. Trying to backpropagate without having cached the forward activations is the most common implementation bug. Memory cost is the price of the chain rule's efficiency.

Exercises

8. Repeat the full backward pass for the same network on input $x = (2, 0)$ and target $y = 0$.

9. Verify the dimensions of every quantity in the worked backward pass against the shapes of $W^{(1)}, W^{(2)}$.
10. Explain, using the recurrence, why a layer whose units are all in ReLU's flat region receives zero gradient and hence does not update.
11. For a network with L layers all of width n, count the matrix multiplications in a forward pass versus a backward pass, and argue backpropagation is the same order of cost as forward evaluation.

4 The softmax function

For **classification** the network must output not a single number but a probability distribution over K classes. The raw output scores — the **logits** — are arbitrary reals; the **softmax** turns them into a valid distribution while preserving their order.

4.1 From scores to probabilities

Worked example — softmax of three logits. Suppose the final layer produces logits $z = (2, 1, 0)$. Exponentiate each and normalize by the total:

$$e^2 \approx 7.39, \quad e^1 \approx 2.72, \quad e^0 = 1, \quad \text{sum} \approx 11.11,$$

$$\hat{y} \approx \frac{7.39, 2.72, 1.00}{11.11} \approx (0.665, 0.245, 0.090).$$

The three outputs are positive and sum to 1 — a probability distribution. The largest logit became the largest probability, but every class keeps a positive share. Exponentiating **before** normalizing (rather than just dividing the logits by their sum) is what guarantees positivity for any real logits, including negative ones.

Definition 8 (Softmax) The **softmax** maps a vector of logits $z \in \mathbb{R}^K$ to $\hat{y} \in \mathbb{R}^K$ by

$$\hat{y}_i = \text{softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}, \quad i = 1, ..., K.$$

Each $\hat{y}_i > 0$ and $\sum_{i=1}^{K} \hat{y}_i = 1$, so $\hat{y}$ is a probability distribution.

Common pitfall Softmax is **not** component-wise: every output $\hat{y}_i$ depends on **all** the logits through the shared denominator. This is the deliberate exception flagged in Chapter 1 — and it is exactly why its derivative (below) is a full Jacobian with off-diagonal terms, not a diagonal gate like ordinary activations.

Remark (shift-invariance and numerical stability) Adding the same constant c to every logit leaves the softmax unchanged: $e^{z_i+c}/\sum_j e^{z_j+c} = e^c e^{z_i}/\left(e^c \sum_j e^{z_j}\right)$, the e^c cancels. In practice one subtracts $c = \max_j z_j$ before exponentiating, so the largest exponent is $e^0 = 1$ and nothing overflows. Same function, safe arithmetic.

4.2 Temperature: sharpening and softening

A single knob rescales how decisively the softmax commits to the top class.

Definition 9 (Temperature) With **temperature** $T > 0$, apply softmax to the scaled logits z/T:

$$\hat{y}_i = \frac{e^{z_i/T}}{\sum_{j=1}^{K} e^{z_j/T}}.$$

As $T \to 0^+$ the distribution concentrates on the largest logit (sharp, near one-hot); as $T \to \infty$ it flattens toward uniform.

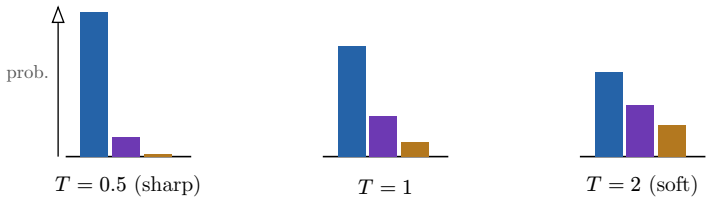

Figure 4: The same logits $z = (2, 1, 0)$ under three temperatures. Low T (left) sharpens toward a one-hot vote for the top class; $T = 1$ is the plain softmax; high T (right) softens toward uniform. Temperature controls confidence without changing which class ranks highest.

4.3 The softmax Jacobian

Because softmax couples all its inputs, its derivative is a $K \times K$ matrix.

Proposition 10 (Softmax Jacobian)

$$\frac{\partial \hat{y}_i}{\partial z_j} = \hat{y}_i(\delta_{ij} - \hat{y}_j), \quad \text{in matrix form} \quad J = \operatorname{diag}(\hat{y}) - \hat{y}\hat{y}^T,$$

where δ_{ij} is the Kronecker delta (1 if $i = j$, else 0).

Proof. Write $S = \sum_k e^{z_k}$, so $\hat{y}_i = e^{z_i}/S$. For the **diagonal** ($i = j$), the quotient rule gives

$$\frac{\partial \hat{y}_i}{\partial z_i} = \frac{e^{z_i} S - e^{z_i} e^{z_i}}{S^2} = \hat{y}_i(1 - \hat{y}_i).$$

For the **off-diagonal** ($i \neq j$), only the denominator depends on z_j:

$$\frac{\partial \hat{y}_i}{\partial z_j} = -\frac{e^{z_i} e^{z_j}}{S^2} = -\hat{y}_i \hat{y}_j.$$

Both cases combine into $\hat{y}_i(\delta_{ij} - \hat{y}_j)$; collecting the entries gives $J = \operatorname{diag}(\hat{y}) - \hat{y}\hat{y}^T$. □

This Jacobian looks like it will make the output-layer gradient messy. The next chapter shows the opposite: paired with the right loss, almost all of it cancels.

Exercises

12. Compute the softmax of $z = (0, 0, 0)$ and of $z = (10, 0, 0)$, and comment on the two extremes.

13. Verify shift-invariance numerically: compute softmax of $(2, 1, 0)$ and of $(5, 4, 3)$.

14. For $z = (2, 1, 0)$, write out the full 3×3 Jacobian $\operatorname{diag}(\hat{y}) - \hat{y}\hat{y}^T$ using $\hat{y} \approx (0.665, 0.245, 0.090)$.

15. Show that each row of the softmax Jacobian sums to 0, and explain this via shift-invariance.

5 Cross-entropy loss

We have a predicted distribution $\hat{y}$; we need a loss measuring how wrong it is against the true label y. **Cross-entropy** is that loss, and it is not an arbitrary choice — it is exactly what **maximum likelihood** (MAT31-2) prescribes. Paired with softmax, its gradient collapses to something beautifully simple.

5.1 Measuring a wrong distribution

Definition 11 (Cross-entropy) For a true distribution y and predicted distribution $\hat{y}$ over K classes, the **cross-entropy loss** is

$$L = -\sum_{i=1}^{K} y_i \log \hat{y}_i.$$

When the label is a single class c — so y is the one-hot vector with $y_c = 1$ — this collapses to $L = -\log \hat{y}_c$.

Worked example — the loss penalizes underconfidence in the truth. Suppose the true class is $c = 1$ and the model predicts $\hat{y} = (0.665, 0.245, 0.090)$. Then $L = -\log 0.665 \approx 0.41$. If instead the model had predicted $\hat{y}_1 = 0.95$, the loss would be $-\log 0.95 \approx 0.05$ — much smaller. And if it had collapsed to $\hat{y}_1 = 0.01$, the loss would be $-\log 0.01 \approx 4.6$, large. Cross-entropy looks **only** at the probability assigned to the correct class and grows without bound as that probability heads to 0.

The loss is small exactly when $\hat{y}$ concentrates on the correct class and diverges as $\hat{y}_c \to 0$ — the network is punished, severely, for being confidently wrong.

5.2 Why this loss: maximum likelihood

Proposition 12 (Cross-entropy is negative log-likelihood) For a single labelled example with true class c, the model assigns probability $\hat{y}_c$ to the observed label, so its log-likelihood is $\log \hat{y}_c = -L$. For a dataset of independent examples,

$$\sum_{\text{examples}} \log \hat{y}_c = -\sum_{\text{examples}} L.$$

Minimizing total cross-entropy is therefore **identical** to maximizing the likelihood of the observed labels.

Remark This is the bridge back to MAT31-2. We did not invent cross-entropy to make the gradient pretty; it **is** the negative log-likelihood of the data under the model's predicted distribution. Training a classifier by minimizing cross-entropy is maximum-likelihood estimation, the same principle you used to fit distributions to data — now applied to a model with millions of parameters.

5.3 The elegant cancellation

Here is the payoff. Composing softmax with cross-entropy, the loss gradient with respect to the **logits** is as simple as it could possibly be.

Theorem 13 (Softmax + cross-entropy gradient) Let $\hat{y} = \mathrm{softmax}(z)$ and $L = -\sum_i y_i \log \hat{y}_i$ with y a probability vector. Then

$$\frac{\partial L}{\partial z} = \hat{y} - y.$$

Predicted minus true — nothing else survives.

Proof. By the chain rule, $\partial L/\partial z_j = \sum_i (\partial L/\partial \hat{y}_i)(\partial \hat{y}_i/\partial z_j)$. The loss derivative is $\partial L/\partial \hat{y}_i = -y_i/\hat{y}_i$, and the softmax Jacobian is $\partial \hat{y}_i/\partial z_j = \hat{y}_i(\delta_{ij} - \hat{y}_j)$. Multiply:

$$\frac{\partial L}{\partial z_j} = -\sum_i \frac{y_i}{\hat{y}_i}\,\hat{y}_i(\delta_{ij} - \hat{y}_j) = -\sum_i y_i(\delta_{ij} - \hat{y}_j) = -y_j + \hat{y}_j \sum_i y_i.$$

The factor $\hat{y}_i$ in the Jacobian **cancels** the $\hat{y}_i$ in the denominator — this is the cancellation. Since $\sum_i y_i = 1$, the result is $\hat{y}_j - y_j$. □

Worked example — the output error, concretely. With logits giving $\hat{y} = (0.665, 0.245, 0.090)$ and true class $c = 1$ (one-hot $y = (1, 0, 0)$),

$$\frac{\partial L}{\partial z} = \hat{y} - y = (-0.335, 0.245, 0.090).$$

Read it directly: the correct class gets a **negative** gradient component (push its logit **up**), the wrong classes get **positive** components (push their logits **down**). The optimizer needs no further computation to start the backward pass.

Remark Because of the cancellation, when a network ends in softmax-with-cross-entropy the output error is simply $\delta^{(L)} = \hat{y} - y$. There is no separate σ' gate to apply — the softmax Jacobian and the loss derivative have already combined. This single fact is why essentially every classifier is trained this way.

Common pitfall The clean form $\hat{y} - y$ holds **only** for the softmax–cross-entropy pair. Use a mismatched loss (say squared error on softmax outputs) and the cancellation fails: the gradient keeps the full Jacobian and can even **vanish** when the model is confidently wrong, stalling training. The elegance is a property of the pair, not of softmax alone.

Exercises

16. For true class $c = 2$ and $\hat{y} = (0.2, 0.5, 0.3)$, compute L and the output error $\hat{y} - y$.
17. Show that $L = -\log \hat{y}_c \geq 0$ always, with equality only when $\hat{y}_c = 1$.
18. Verify the cancellation by computing $\partial L/\partial z$ directly for $K = 2$ and confirming it equals $\hat{y} - y$.
19. Two independent examples have losses $-\log 0.8$ and $-\log 0.6$. Write the total log-likelihood of the pair and relate it to the summed loss.

6 Residual connections

Very deep networks are hard to train, in part because the error must survive multiplication through dozens of layers (Chapter 8). **Residual connections** are a structural fix so simple it is almost embarrassing: let each block add its output to its input instead of replacing it.

6.1 Identity plus a perturbation

Definition 14 (Residual connection) A **residual** (or **skip**) connection replaces a block f by

$$a = x + f(x).$$

The block no longer has to produce the whole next representation; it only learns the **perturbation** $f(x)$ added to the identity.

Worked example — learning nothing is now easy. Suppose the ideal thing for a block to do is **leave its input unchanged**. A plain block $a = f(x)$ must laboriously learn $f(x) = x$ — the identity — across all its weights. A residual block $a = x + f(x)$ achieves the same by simply driving $f(x) \to 0$, i.e. sending its weights toward zero, which is far easier. Residual blocks make the identity the **default**, and learn only departures from it.

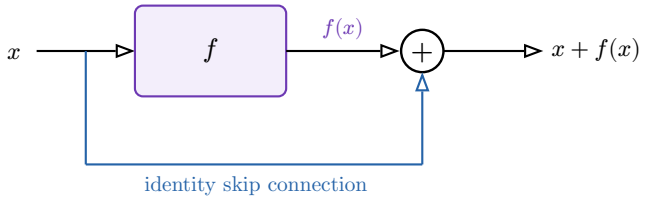

Figure 5: A residual block. The input x flows through the block f **and** along a skip path that adds it back unchanged, producing $x + f(x)$. The block only has to learn the correction $f(x)$.

6.2 Why gradients flow

The training benefit shows up in the Jacobian.

Proposition 15 (Residual Jacobian) For $a = x + f(x)$,

$$\frac{\partial a}{\partial x} = I + \frac{\partial f}{\partial x}.$$

Proof. Differentiate $a = x + f(x)$ component-wise: the identity term contributes the identity matrix I, and f contributes its own Jacobian. Linearity of the derivative gives the sum. □

The additive identity I is the whole point. Backpropagating through a stack of residual blocks multiplies factors of the form $I + \partial f/\partial x$, **each of which contains a clean I**. Even if the $\partial f/\partial x$ terms are tiny, the gradient still flows through the I — it cannot decay to zero merely by passing through many layers, as it would if each factor were $\partial f/\partial x$ alone.

Remark This is the mathematical reason networks went from tens of layers to hundreds. The skip connection provides a "gradient highway": a path along which the error reaches early layers undiminished. We will see in Chapter 8 precisely what catastrophe it averts.

Exercises

20. For $f(x) = Wx$ (a linear block), write the Jacobian of $x + f(x)$ and contrast it with that of the plain block f.

21. Stack two residual blocks, $a = x + f(x)$ then $b = a + g(a)$. Expand $\partial b/\partial x$ and identify the term that survives even when both inner Jacobians are near zero.

22. Explain why initializing a residual block's weights so that $f(x) \approx 0$ makes the untrained block behave like the identity, and why that is a safe starting point.

7 Layer normalization

A second structural stabilizer attacks a different problem: as signals pass through many layers, the **scale** of activations can drift — growing or shrinking until activations saturate or explode. **Layer normalization** rescales each layer's vector back to a controlled mean and variance.

7.1 Standardizing a single example's activations

Worked example — rescaling a drifting vector. Suppose a layer produces $z = (10, 14, 12)$ — all large, with mean 12. Its components measured from the mean are $(-2, 2, 0)$, with variance $s^2 = \frac{4+4+0}{3} = \frac{8}{3}$ and $s \approx 1.63$. Standardizing,

$$\hat{z} = \frac{z - 12}{1.63} \approx (-1.22, 1.22, 0).$$

Whatever the incoming scale, $\hat{z}$ now has mean 0 and variance 1. The **shape** of the activation pattern is preserved; only its location and spread are reset.

Definition 16 (Layer normalization) For a pre-activation vector $z \in \mathbb{R}^n$, layer normalization computes the mean and variance **across the components of that single vector**,

$$\mu = \frac{1}{n}\sum_{i=1}^{n} z_i, \quad s^2 = \frac{1}{n}\sum_{i=1}^{n} (z_i - \mu)^2,$$

standardizes,

$$\hat{z}_i = \frac{z_i - \mu}{\sqrt{s^2 + \varepsilon}},$$

and then applies a learned per-component scale γ_i and shift β_i:

$$\mathrm{LN}(z)_i = \gamma_i \hat{z}_i + \beta_i.$$

The constant $\varepsilon > 0$ prevents division by zero.

By forcing every normalized layer to mean 0 and unit variance before the learned γ, β, the activations are held on a predictable scale throughout the network, which keeps them out of the saturating tails of activations and makes training markedly more stable.

Remark The learned γ and β matter: pure standardization would rob the layer of the ability to use other scales, so the network is given back explicit control to **rescale and reshift** if that helps. Normalization sets a sensible default (mean 0, variance 1); γ, β let the network deviate from it deliberately rather than by accident.

7.2 Layer versus batch normalization

Proposition 17 (Which axis is normalized) Layer normalization computes μ and s^2 across the **features of one example**. **Batch normalization** computes them, per feature, across the **examples in a batch**.

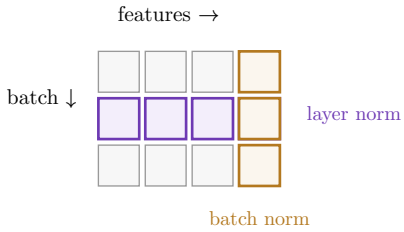

Figure 6: Each cell is one activation; rows are examples in a batch, columns are features. **Layer normalization** (purple row) averages across the features of a single example. **Batch normalization** (orange column) averages across the batch for a single feature. Layer norm needs no batch; batch norm does.

Common pitfall Because layer normalization never looks across the batch, it behaves **identically** whether the batch has one example or a thousand, and is unchanged between training and inference. Batch normalization depends on batch composition and needs running statistics at test time — a frequent source of train/test discrepancies. This batch-independence is the main reason layer norm is the default in the sequence models you will meet in S6.

Exercises

23. Apply layer normalization (ignore ε, $\gamma = 1$, $\beta = 0$) to $z = (1, 2, 3)$.

24. Show that layer normalization is invariant to adding a constant to all components of z, and to multiplying them all by a positive constant (before γ, β).

25. Explain why setting $\gamma_i = \sqrt{s^2 + \varepsilon}$ and $\beta_i = \mu$ would exactly undo the normalization, and why the network is nonetheless rarely served by doing so.

26. Give a concrete scenario (e.g. a batch of size 1) where batch normalization is ill-defined but layer normalization works fine.

8 Vanishing and exploding gradients

We can now name precisely the disease that residual connections, careful initialization, and normalization were all invented to treat. It is a direct consequence of the backward recurrence: **gradients are products of many factors, and products of many numbers either explode or vanish unless those numbers sit near 1.**

8.1 Compounding through depth

The recurrence $\delta^{(l)} = \left(\left(W^{(l+1)}\right)^T \delta^{(l+1)}\right) \odot \sigma'\left(z^{(l)}\right)$ multiplies the error by a factor involving $W^{(l+1)}$ and σ' at **every** layer. Tracing the error from the output back to an early layer therefore multiplies L such factors together.

Proposition 18 (Geometric compounding) If each backward step scales the gradient's magnitude by roughly a constant factor κ (set by the typical size of the weights and of σ'), then the gradient reaching a layer L steps from the output scales like κ^L. Hence

- $\kappa < 1$ (small weights, saturating σ') $\Longrightarrow \kappa^L \to 0$: **vanishing gradients**;
- $\kappa > 1$ (large weights) $\Longrightarrow \kappa^L \to \infty$: **exploding gradients**.

Worked example — the tyranny of exponents. Take a modest-looking $\kappa = 0.8$ and a depth of $L = 50$. Then $\kappa^L = 0.8^{50} \approx 1.4 \times 10^{-5}$: the gradient at the first layer is some **hundred-thousand times** smaller than at the last — those early layers barely move. Flip to $\kappa = 1.2$: $1.2^{50} \approx 9100$, and the gradient explodes by four orders of magnitude, sending the optimizer careening. Only κ very close to 1 keeps κ^L sane across deep stacks.

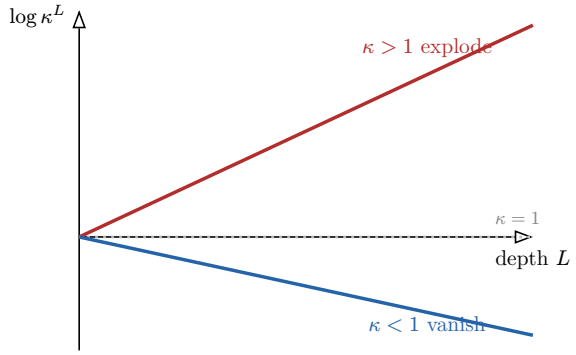

Figure 7: The (log) magnitude of the backpropagated gradient against depth. For $\kappa > 1$ it grows linearly in log-scale — exponential explosion. For $\kappa < 1$ it falls away — exponential vanishing. Only $\kappa = 1$ (flat) keeps early and late layers on the same footing.

8.2 The connection to eigenvalues

Remark "Multiply by a constant κ" is the scalar caricature; the honest statement is about the matrices $\left(W^{(l)}\right)^T$ being multiplied in sequence. Whether their product grows or shrinks is governed by their **eigenvalues** (MAT21-1): repeated multiplication amplifies directions with eigenvalue magnitude > 1 and crushes those with magnitude < 1 — exactly the A^k behaviour you analysed for matrix powers. Vanishing/exploding gradients are that phenomenon, run backward through the network.

8.3 Mitigations, and why they work

Keeping κ near 1.

- **Residual connections.** Each factor becomes $I + \partial f/\partial x$, whose I keeps a clean 1 in the product — the gradient cannot vanish merely by depth (Chapter 6).
- **Careful initialization.** Choose the initial weight scale so that each backward factor has magnitude ≈ 1 (the idea behind Xavier/He initialization): set the variance of the weights from the layer widths so signals neither amplify nor decay on average.
- **Normalization.** Re-centering and re-scaling activations (Chapter 7) keeps them out of the saturating tails where $\sigma' \approx 0$, which would otherwise force κ toward 0.

Common pitfall Depth is not free. A network that trains perfectly at 10 layers can fail completely at 100 with no change but depth, purely because κ^L has crossed from "manageable" to "catastrophic". The mitigations above are not optional polish — without them, naively stacking layers makes networks **worse**, which for years was the central obstacle of the field.

Exercises

27. Compute κ^L for $\kappa \in \{0.9, 1.0, 1.1\}$ at $L = 30$ and comment.

28. A backward factor is $\kappa = m \cdot s$ where m is a typical weight magnitude and $s = \max \sigma'$. For the sigmoid, $\max \sigma' = \frac{1}{4}$. What weight magnitude m is needed to reach $\kappa = 1$, and what does that suggest about sigmoids in deep networks?

29. Using the eigenvalue picture, explain why a repeated weight matrix with all eigenvalues of magnitude exactly 1 (e.g. orthogonal) is a natural target for initialization.

30. Argue that residual connections change the effective κ from $\partial f/\partial x$ to something with a guaranteed 1, and why this helps even when f is badly scaled.

9 Regularization

A network with millions of parameters can memorize its training data, fitting even the noise — and then generalize poorly to new data. This is **overfitting**. **Regularization** deliberately constrains the model to fit less perfectly in exchange for generalizing better. We connect both standard methods to the **bias–variance tradeoff**.

9.1 Overfitting and the bias–variance tradeoff

Worked example — memorizing versus learning. Imagine fitting noisy points that genuinely lie near a straight line. A high-capacity network can wiggle through **every** point exactly, including the noise — zero training error, but a wildly oscillating function that predicts badly between the points. A constrained model that draws a clean line has higher training error yet predicts new points far better. The first **overfits** (low bias, high variance); the second trades a little bias for much less variance.

Remark (the tradeoff) Prediction error splits into **bias** (error from the model being too simple to capture the truth) and **variance** (error from the model being so flexible it chases noise). An unregularized network sits at low bias, high variance. Regularization adds a little bias to cut variance; the best setting **minimizes their sum**, the generalization error.

9.2 Weight decay (L2 regularization)

Definition 19 (Weight decay) **Weight decay** adds a penalty proportional to the squared norm of the weights:

$$L_{\text{reg}} = L + \frac{\lambda}{2} \sum_l \left\| W^{(l)} \right\|^2, \quad \lambda > 0.$$

Its gradient contribution is $\lambda W^{(l)}$, so a gradient step becomes

$$W^{(l)} \leftarrow W^{(l)} - \eta \frac{\partial L}{\partial W^{(l)}} - \eta \lambda W^{(l)}.$$

The extra term $-\eta\lambda W^{(l)}$ shrinks every weight toward 0 at each step — hence the name **decay**. Penalizing large weights favours smaller-magnitude parameters, which produce smoother, less oscillatory functions less able to fit noise: bias up a little, variance down a lot.

Common pitfall Larger λ is not uniformly better. Push λ too high and the penalty dominates the data term, driving weights toward 0 and the model toward underfitting — high bias now. Weight decay is a **dial** on the bias–variance tradeoff, with a sweet spot, not a setting to maximize.

9.3 Dropout

Definition 20 (Dropout) During training, **dropout** sets each activation to 0 independently with probability p (per unit, per example), and rescales the survivors so their expected value is unchanged. At test time no units are dropped.

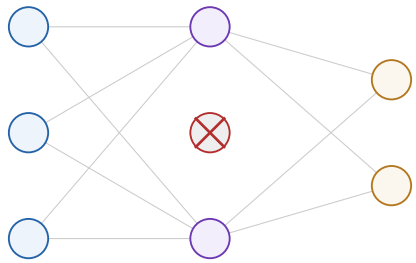

Figure 8: One dropout step. A randomly chosen hidden unit (red ✗) is zeroed for this example, so the network must produce its output without it. A different random subset is dropped each step, preventing any unit from relying on a specific other.

Dropout stops units from **co-adapting** — relying on the presence of specific other units — because any unit may vanish on any step. The network is forced to spread its representation across redundant features, which is more robust and generalizes better. It behaves like training a vast ensemble of sub-networks that share weights.

Common pitfall Do not forget the rescaling, and do not drop at test time. During training the survivors are divided by $(1-p)$ so that the **expected** activation matches the no-dropout case; at test time all units are present and nothing is scaled. Skip the rescaling, or leave dropout on at inference, and the activation magnitudes no longer match what the next layer was trained to expect.

Remark Both methods are knobs on the same tradeoff. Increasing λ (weight decay) or p (dropout) moves the model toward higher bias and lower variance; the value minimizing **validation** error balances the two. They address overfitting from complementary angles — weight decay shrinks parameter magnitudes, dropout disrupts co-adaptation — and are routinely used together.

Exercises

31. Show that the weight-decay update can be written $W^{(l)} \leftarrow (1 - \eta\lambda)W^{(l)} - \eta\,\partial L/\partial W^{(l)}$, and identify the multiplicative shrink factor.

32. With dropout probability $p = 0.5$, by what factor must surviving activations be rescaled so the expected activation is preserved?

33. Sketch, qualitatively, training and validation error against model complexity, and mark where bias dominates, where variance dominates, and the generalization optimum.

34. Argue why a very large λ and a very large p both push a network toward underfitting, despite acting through different mechanisms.